

Unpenalized Magnitude Recovery for Signed Biological Networks

Stage (c): De-biased Refit via Bound-Constrained Optimization

Algorithm Specification

July 3, 2026

Abstract

This document specifies a follow-on “Stage (c)” algorithm designed to recover unbiased edge magnitudes in a multi-state Poisson GLM for spiking neural networks. While the preceding EM-FDR-Bagging stage robustly identifies the topological support of the network (detecting true edges and assigning Dale’s principle signs), its use of L_1 regularization systematically shrinks edge magnitudes. In addition, repeated enforcement of a spectral-radius bound can further scale down the fitted connectivity matrix when the unconstrained update would exceed the stability limit. To isolate magnitude estimation from structural discovery, we define a bound-constrained optimization task that freezes the discovered topological mask, removes the sparsity penalty, and enforces biological signs. The recommended implementation fits compact active-parameter vectors rather than dense mostly-zero matrices: signed Dale-constrained entries are represented by squared parameters, free entries are copied directly, and the full matrix A is rebuilt in the forward pass.

1 Introduction and Background

1.1 Input Data and Model Summary

The input consists of a discrete spike-train recording of N neurons over T time bins of width Δt , yielding a count matrix $Y \in \mathbb{R}^{T \times N}$. The system is modeled as a non-stationary, multi-state Poisson Generalized Linear Model (GLM). The conditional firing rate for neuron i at time t depends on a shared connectivity matrix $A \in \mathbb{R}^{N \times N}$ and a state-dependent bias vector $B_{\hat{S}_t}$:

$$\lambda_t = \exp \left(\min(A Y_{t-1} + B_{\hat{S}_t}, \eta_{\text{clip}}) \right) \Delta t, \quad Y_t \sim \text{Poisson}(\lambda_t) \quad (1)$$

where $\hat{S}_{1:T}$ is the hard-decoded state sequence used by the M-step, and η_{clip} prevents numerical overflow. In locked mode $\hat{S}_{1:T}$ is copied from Stage (b); in refit mode it is updated by the Stage (c) E-step and Viterbi decode before the following de-biased M-step.

1.2 The Preceding FDR-Bagging Stage

Prior to this stage, an EM-FDR-Bagging routine was run. It drew subsamples of lag pairs and fit the model using an L_1 penalty (Lasso shrinkage) to suppress noise. Edges that survived a source-pooled null hypothesis across multiple bags were retained. Furthermore, source neurons were classified based on their total outgoing edge sum: strictly positive columns were labeled Excitatory, strictly negative columns were labeled Inhibitory, and intermediate columns were labeled Undecided.

The output of the bagging stage provides:

1. A vetted topological mask indicating which edges are statistically significant.

2. A categorical label for each source node: *Excitatory*, *Inhibitory*, or *Undecided*.
3. Approximate, heavily L_1 -shrunk point estimates for the connectivity ($\bar{A}$) and state biases ($\bar{B}$).

2 Motivation: The Need for a De-biased Refit

Because the FDR-Bagging stage relies on an L_1 penalty to identify structural zeros, all surviving non-zero edges in $\bar{A}$ are systematically biased toward zero. This is a well-known property of Lasso regression. The shrinkage is useful during edge discovery because it suppresses noisy weak coefficients, but it is undesirable once the support has already been selected.

There is a second, model-specific source of underestimation. PRISM-EM also enforces a spectral-radius limit, $\rho(A) \leq \rho_{\max}$, to keep the recurrent GLM numerically stable. If an M-step update produces a matrix with $\rho(A) > \rho_{\max}$, the projection rescales the effective matrix. This operation preserves the discovered edge pattern and signs, but it can reduce many edge magnitudes at once. Therefore, the Stage (b) aggregate $\bar{A}$ should be interpreted as a conservative, regularized support estimate, not as a final magnitude estimate.

To recover the biological magnitudes of these connections, we perform a “de-biased Lasso” step: an unpenalized Maximum Likelihood Estimation (MLE) refit restricted to the selected topological support. The simplest mode keeps the Stage (b) state sequence locked, matching the FDR-bagging refits; the multi-state mode may instead alternate de-biased M-steps with state refitting.

The state biases should be re-fitted in the same optimization. In Eq. (1), AY_{t-1} and $B_{\hat{S}_t}$ enter the log-rate additively, so residual coupling between the connectivity and the state-dependent baseline can remain after the support-discovery stage. Once the magnitudes in A are allowed to expand without the L_1 penalty, the optimum values of B may also shift.

This is a bound-constrained optimization problem. We must maximize the Poisson log-likelihood while enforcing four strict rules:

1. **Structural Zeros:** Edges pruned in the bagging stage must remain exactly zero.
2. **Dale’s Principle (Strict):** Excitatory nodes must have strictly non-negative outgoing edges. Inhibitory nodes must have strictly non-positive outgoing edges.
3. **Dale’s Principle (Relaxed):** Edges originating from “undecided” nodes that survived pruning must be free to adjust their sign as dictated by the unpenalized data.
4. **Free Diagonal:** The diagonal elements A_{ii} are not treated as tested biological edges and must be left free to vary. They therefore belong to the free parameter set, even when neuron i has an excitatory or inhibitory Dale label.

3 Methodology: Active-Set Parameter Transformation

To enforce the above constraints without relying on manual clamping (which can disrupt the momentum-based Adam optimizer), we use a parameter transformation for A : the *squared-parameter trick*. The complete trainable variable set is (u, v, B) : the active connectivity vectors defined below plus the state-bias matrix. The biases are re-fitted jointly with A and are initialized from $\bar{B}$, but they do not need the Dale sign transform. The masks below define the mathematical structure of A , but the optimizer should not have to carry full $N \times N$ tensors full of zeros.

We construct two static, mutually exclusive structural masks based on the FDR-bagging output:

- $M_{\text{Dale}} \in \{-1, 0, 1\}^{N \times N}$: This mask handles off-diagonal surviving edges from signed source nodes. It contains +1 for surviving excitatory edges, -1 for surviving inhibitory edges, and 0 otherwise.
- $M_{\text{Free}} \in \{0, 1\}^{N \times N}$: This mask handles entries whose sign is not Dale-constrained. It contains 1 for surviving off-diagonal edges from undecided source nodes, 1 for every diagonal element A_{ii} , and 0 otherwise.

The dense mathematical form is

$$A = (M_{\text{Dale}} \odot U^{\odot 2}) + (M_{\text{Free}} \odot V) \quad (2)$$

where $\odot$ denotes element-wise multiplication (Hadamard product), and $U^{\odot 2}$ denotes the element-wise square of U . This form is useful for reasoning about the constraints. For implementation, however, it is better to fit active vectors:

$$\mathcal{D} = \{(i, j) : (M_{\text{Dale}})_{ij} \neq 0\}, \quad \mathcal{F} = \{(i, j) : (M_{\text{Free}})_{ij} = 1\}, \quad (3)$$

$$u \in \mathbb{R}^{|\mathcal{D}|}, \quad v \in \mathbb{R}^{|\mathcal{F}|}. \quad (4)$$

In each forward pass, initialize A as an all-zero matrix and scatter the active parameters into it:

$$A_{ij} = \begin{cases} (M_{\text{Dale}})_{ij} u_k^2, & (i, j) = \mathcal{D}_k, \\ v_\ell, & (i, j) = \mathcal{F}_\ell, \\ 0, & \text{otherwise.} \end{cases} \quad (5)$$

Thus each free entry, including every diagonal A_{ii} , is copied one-to-one from v , while each Dale-constrained edge is generated by squaring the corresponding element of u and applying the fixed sign.

Why this works: Because u is squared, u_k^2 is non-negative. Multiplying it by the sign stored in M_{Dale} enforces excitatory or inhibitory signs by construction. Meanwhile, the unconstrained vector v allows undecided off-diagonal edges and diagonal self-history terms to explore positive or negative values freely. The vector formulation is also cleaner numerically: pruned edges have no optimizer state, receive no stale momentum, and cannot accidentally re-enter through implementation mistakes.

4 Fitting Details and Safeguards

4.1 Vanishing Gradients and Initialization

The primary pitfall of the squared-parameter trick lies in the chain rule of backpropagation. The gradient of the loss $\mathcal{L}$ with respect to a parameter U_{ij} is:

$$\frac{\partial \mathcal{L}}{\partial U_{ij}} = \frac{\partial \mathcal{L}}{\partial A_{ij}} \cdot 2U_{ij} \cdot (M_{\text{Dale}})_{ij} \quad (6)$$

In the active-vector notation this becomes

$$\frac{\partial \mathcal{L}}{\partial u_k} = \frac{\partial \mathcal{L}}{\partial A_{ij}} \cdot 2u_k \cdot (M_{\text{Dale}})_{ij}, \quad (i, j) = \mathcal{D}_k. \quad (7)$$

If the optimizer pushes u_k to exactly zero, or initializes it very close to zero, the gradient vanishes entirely. The parameter becomes “stuck” and cannot recover, effectively acting as an unintended structural pruning.

Mitigation via Initialization: To prevent vanishing gradients, initialize the learnable signed-edge parameters securely away from zero, using the approximate shrunk values ($\bar{A}$) from the FDR-bagging stage.

$$\begin{aligned} u_k^{\text{init}} &= \max\left(\sqrt{|\bar{A}_{ij}|}, \epsilon_u\right), & (i, j) &= \mathcal{D}_k, \\ v_\ell^{\text{init}} &= \bar{A}_{ij}, & (i, j) &= \mathcal{F}_\ell. \end{aligned} \tag{8}$$

Here ϵ_u is a small positive floor used only for Dale-constrained parameters whose bagged estimate is extremely close to zero. The bias vectors are initialized directly as $B^{\text{init}} = \bar{B}$.

4.2 Spectral-Radius Projection

To maintain numerical stability in the Poisson GLM, the effective matrix A must not exceed a maximum spectral radius $\rho_{\max}$ (typically 0.95).

In a standard network, one applies a scalar scaling factor to A . In our transformed parameter space, we must apply the scaling to the underlying learnable variables u and v such that the resulting effective A is correctly scaled.

At the configured spectral-projection interval during the M-step, we construct the effective A and calculate its spectral radius $\rho(A)$. If $\rho(A) > \rho_{\max}$, we define a scaling factor:

$$\begin{aligned} \gamma &= \frac{\rho_{\max}}{\rho(A)}, \\ u &\leftarrow u \cdot \sqrt{\gamma}, \\ v &\leftarrow v \cdot \gamma. \end{aligned} \tag{9}$$

We then update the underlying active parameters *in-place* according to Eq. (9). Because the forward pass squares u , scaling u by $\sqrt{\gamma}$ correctly results in the corresponding effective Dale-constrained elements of A being scaled by exactly γ . Scaling v by γ does the same for free entries. Structural zeros are unaffected, diagonal terms remain free, and biological signs cannot flip. The spectral constraint is therefore compatible with the sign constraints, although it can still limit final magnitudes when the unconstrained MLE wants a matrix outside the stable region.

5 Implementation Plan for `prism_deBiasFit3c.py`

The production implementation should be a PyTorch distributed trainer, launched the same way as the existing Stage (a) and Stage (b) jobs:

```
module load pytorch
torchrun --standalone --nnodes=1 --nproc_per_node=4 \
  ./prism_deBiasFit3c.py \
  --basePath <basePath> \
  --fdrFitName <stageB_aggregate_name> \
  --outFitName <stageC_output_name>
```

On Perlmutter this uses the four A100 GPUs assigned to one node. The script should reuse distributed utilities from `PrismEM_Workhorse3c.py` (`init_distributed`, `broadcasts`, `barriers`, `seeding`,

pair loaders, E-step helpers, simplex projection, and Viterbi decoding). The dense unconstrained `SwitchingPoissonGLM`, `run_full_fit`, and `run_locked_mstep` should not be reused directly because Stage (c) needs the active-vector parameterization (u, v, B) and no L_1 pruning. If the implementation becomes long, put the Stage (c) model and training loop in `PrismDeBias.Workhorse3c.py`, leaving `prism_deBiasFit3c.py` as the command-line, I/O, and provenance wrapper.

5.1 Inputs and Initialization

The primary input is the Stage (b) aggregate fit `<basePath>/prismFit/<fdrFitName>.prismEM.npz`. The script should recover the original spike-file stem and the reference EM fit name from the existing metadata provenance, rather than asking the user to provide multiple file names. It should then load the source spikes from `spikesData/`, slice the same `time_range_bins`, and verify that the spike window length matches the Stage (b) `S_hat` and `c_hat` arrays.

The Stage (c) support and initialization are:

- Use Stage (b) `selected_mask` for the off-diagonal support.
- Use Stage (b) `neuron_type` for Dale-constrained source columns.
- Put all diagonal entries into the free set $\mathcal{F}$.
- Initialize Dale-constrained magnitudes from Stage (b) `A_hat`: $u_k = \max(\sqrt{|A_{ij}^{(b)}|}, \epsilon_u)$, with the sign supplied only by M_{Dale} .
- Initialize free off-diagonal entries and all diagonal entries from Stage (b) `A_hat`: $v_\ell = A_{ij}^{(b)}$.
- Initialize the bias matrix from Stage (b) `B_hat`.

Stage (b) `A_prune` must be copied to the output unchanged for comparison, but it is not the fitted Stage (c) matrix.

5.2 State Handling

Expose a three-way command-line option:

- `--state_mode locked`: keep Stage (b) states fixed and fit only (u, v, B) .
- `--state_mode refit`: alternate E-step updates of c_t and Viterbi decoding with de-biased M-step updates of (u, v, B) .
- `--state_mode auto`: use `refit` when `num_states>1`, otherwise use `locked`.

All M-step minibatches should use destination-bin state labels: S_t for the lag pair (Y_{t-1}, Y_t) . This matches the Stage (b) FDR-bagging convention and the rate model in Eq. (1).

5.3 Training Hyperparameters

The script should read defaults from the Stage (b) and reference EM metadata, but expose the important controls explicitly:

- `--num_debias_iters`: outer E/M iterations for `state_mode refit`.
- `--warmup_locked_epochs`: optional fixed-state M-step epochs before state refitting starts.

- `--m_epochs`, `--batch_size`, `--lr_mstep`, and `--lr_end_factor`: Adam M-step controls.
- `--pgd_iter`, `--lr_estep`, and `--lambda2`: E-step controls used only when states are refit. The parameter `--lambda2` is the temporal smoothness weight on the soft state probabilities,

$$\lambda_2 \sum_t \|c_t - c_{t-1}\|_2^2.$$

In the projected-gradient E-step this contributes the gradient term $2\lambda_2(c_t - c_{t-1})$. Larger values make the inferred state probabilities change more slowly over time; smaller values allow more rapid state switching. It does not regularize A or B directly. In `state_mode locked`, the states are copied from Stage (b), the E-step is skipped, and `--lambda2` is effectively irrelevant.

- `--rho_max`, `--prescale_m_step_4_ArhoMax`, `--delay_iter_4_ArhoMax`, and `--target_iter_4_ArhoMax`: spectral-radius controls.
- `--eps_u` and `--seed`: numerical floor and RNG seed.

There is no Stage (c) L_1 penalty; record `lambda3=0` in metadata.

5.4 Output Schema and Provenance

The output should be written to `<basePath>/prismFit/<outFitName>.prismEM.npz`. Preserve the original Stage (b) arrays for comparison and add unambiguous Stage (c) arrays:

- Keep Stage (b) `A_hat`, `A_prune`, and `B_hat` unchanged.
- Save the direct Stage (c) fit as `A_debias` and `B_debias`.
- Save the original Stage (b) states as `S_stageB`, `c_stageB`, and `S_stageB.CL`.
- Save final display states as `S_hat`, `c_hat`, and `S_hat.CL`. In locked mode these equal the Stage (b) states; in refit mode they are the final Stage (c) states.
- Save histories for E-step NLL, M-step loss/NLL, spectral radius, spectral correction strength, learning rate, and effective nonzero count.
- Save support diagnostics: Dale/free index counts, diagonal count, selected-edge count, and optional arrays mapping active-vector entries back to (i, j) coordinates.

Do not perform a second post-fit pruning pass on `A_debias`; the purpose is to inspect the direct constrained MLE result.

Metadata should continue the existing provenance scheme. Set `fit_type="prismEM_deBias_stageC"` and add a `deBias_stageC` block containing the program name, input Stage (b) file/name, recovered reference EM name, source spike file/name, `state_mode`, support construction rules, initialization rules, hyperparameters, output name, elapsed time, and the trainable parameter counts. The top-level `provenance` dictionary should retain all Stage (a)/(b) entries and add `deBias_stageC_input`, `deBias_stageC_file`, and `deBias_stageC_outFitName`.

5.5 Plotting Compatibility

`prism_EM_eval3c.py` should detect Stage (c) files from `fit_type` or the presence of `deBias_stageC`. For Stage (c) outputs it should use `A_debias/B_debias` as the primary display matrices and include “Stage (c) de-biased refit” in plot titles. For Stage (b) files it should keep the current behavior and title plots as “Stage (b) FDR aggregate”. The Stage (c) output should nevertheless retain all relevant arrays so that a dedicated plotter can be built later without rerunning the fit.

6 Algorithm Summary for Implementation

An agent tasked with implementing Stage (c) should follow these explicit steps:

1. **Load Data:** Load the Stage (b) aggregate, recover the source spike file from provenance, slice the Stage (b) time window, and load Stage (b) states and aggregates.
2. **Construct Static Masks and Index Sets:** Instantiate constant tensors M_{Dale} and M_{Free} based on Stage (b) `selected_mask`, `neuron_type`, and the diagonal. Put signed off-diagonal surviving edges in $\mathcal{D}$ and put undecided surviving off-diagonal edges plus all diagonal entries in $\mathcal{F}$.
3. **Initialize Parameters:**
 - $u_k \leftarrow \max(\sqrt{|A_{ij}^{(b)}|}, \epsilon_u)$ for $(i, j) = \mathcal{D}_k$ (requires gradient)
 - $v_\ell \leftarrow A_{ij}^{(b)}$ for $(i, j) = \mathcal{F}_\ell$ (requires gradient)
 - $B \leftarrow B^{(b)}$ (requires gradient)
4. **Training Loop (Minibatch Adam):**
 - *Forward Pass:* Construct a zero matrix A , scatter $(M_{\text{Dale}})_{ij} u_k^2$ into $(i, j) = \mathcal{D}_k$, and scatter v_ℓ into $(i, j) = \mathcal{F}_\ell$.
 - *State Handling:* In locked mode use Stage (b) destination-bin labels. In refit mode update c_t by the E-step, Viterbi-decode S_t , and then use destination-bin labels in the M-step.
 - *Loss:* Compute negative Poisson log-likelihood using A and $B_{\hat{S}_t}$, with no L_1 penalty ($\lambda_3 = 0$).
 - *Backward Pass:* Call `loss.backward()` and `optimizer.step()`.
5. **Spectral Projection (End of interval):**
 - Construct current effective A under `torch.no_grad()`.
 - Compute $\rho(A)$. If $\rho(A) > \rho_{\max}$, compute $\gamma = \rho_{\max}/\rho(A)$.
 - In-place multiply u by $\sqrt{\gamma}$ and v by γ .
6. **Save Output:** Once converged, construct the final effective matrix and save it as `A_debias`; save the jointly fitted biases as `B_debias`. Preserve Stage (b) `A_hat`, `A_prune`, and `B_hat` unchanged for comparison. Do not run a second pruning pass on `A_debias`.